Computer Networks — Course Project Report

TCPlay: A Simulation-Based Study of Transport-Layer Behaviour and Quality of Experience in Adaptive Video Streaming

Case study: "Transport Layer in Practice – How Netflix Streams Videos Using TCP and UDP"

Submitted by

Tarandeep Singh (24BKT0077)

Rohit Pal (24BCE2363)

Aditya Ghadge (24BCE0821)

Course: Computer Networks
Course code: BCSE308L
Faculty: Thamizharasan S

October 2026

Acknowledgement

We express our sincere gratitude to our course instructor and the faculty of the Computer Networks course for their guidance, and to the open standards and research communities (IETF, ACM SIGCOMM and the authors cited in the references) whose publicly available work made this study possible. We also thank our peers for their feedback during the project reviews.

Abstract

Large-scale video streaming must deliver high-bitrate content across paths whose bandwidth, latency, jitter and packet loss vary continuously. Publicly documented accounts of Netflix's Open Connect platform describe bulk video delivery over HTTP(S) on TCP, with adaptive bitrate (ABR) selection performed by the client, while UDP-based transports appear in other roles. This project investigates how transport-layer behaviour propagates into the quality a viewer actually experiences.

We present TCPlay, a seeded, reproducible, time-stepped simulator that chains network conditions, transport behaviour, segmented video delivery, playback buffering, ABR decisions and a simplified quality-of-experience (QoE) score. The transport layer is modelled as a simplified TCP-like protocol (handshake, congestion window, retransmission, head-of-line blocking), a simplified UDP-like protocol (independent datagrams, loss, reordering, optional application-level recovery) and a QUIC-like extension. Throughput-based and buffer-based ABR are implemented, and every bitrate decision is logged with its rule.

Experiments are controlled: transports are compared under identical bandwidth traces, loss draws, jitter, video ladder and seed, so only transport behaviour differs, and each condition is repeated over 30 seeds with 95% confidence intervals. In the default scenario the UDP-like transport without recovery reached the highest bitrate but delivered damaged segments (QoE 38.1), whereas UDP with ARQ scored highest (60.5) against 54.5 for TCP-like delivery; independent QUIC-like streams reduced head-of-line blocking about five-fold; and the TCP-like model matched the Mathis throughput approximation to within about 9% where its assumptions hold. All reported values are measured from simulation output. The project makes no claim to reproduce Netflix's proprietary systems; its purpose is to expose engineering trade-offs, not to declare a universal winner.

Keywords: transport layer, TCP, UDP, QUIC, head-of-line blocking, adaptive bitrate streaming, playback buffer, QoE, network simulation

Table of Contents

1 Introduction

- 1.1 Background and motivation
- 1.2 Case study: how Netflix streams video
- 1.3 Problem statement
- 1.4 Objectives
- 1.5 Scope and contributions
- 1.6 Organisation of the report

2 Background and Literature Review

- 2.1 The transport layer
- 2.2 Throughput under loss: the Mathis model
- 2.3 HTTP adaptive streaming and ABR
- 2.4 Quality of Experience
- 2.5 Related tools and the gap addressed

3 System Design and Architecture

- 3.1 Requirements
- 3.2 Architecture
- 3.3 Conceptual pipeline
- 3.4 Module descriptions
- 3.5 Units and configuration

4 Simulation Models and Methodology

- 4.1 Network model
- 4.2 TCP-like transport
- 4.3 UDP-like transport
- 4.4 QUIC-like transport
- 4.5 Video, buffer and playback
- 4.6 Adaptive bitrate algorithms
- 4.7 QoE model
- 4.8 Session algorithm
- 4.9 What is simulated and what is not

5 Implementation

- 5.1 Technology stack
- 5.2 Backend and API
- 5.3 Command-line interface
- 5.4 Dashboard
- 5.5 Reproducibility and export

6 Experimental Design

- 6.1 Research questions
- 6.2 Default scenario
- 6.3 Experiments
- 6.4 Statistical method
- 6.5 Validation against theory

7 Results and Discussion

- 7.1 Application walkthrough with sample inputs

7.2 Experiment 1: TCP-like vs UDP-like vs QUIC-like
7.3 Experiment 2: TCP-like throughput vs the Mathis approximation
7.4 Experiment 3: repeated trials
7.5 Experiment 4: throughput-based vs buffer-based ABR
7.6 Experiment 5: sensitivity analysis
7.7 Failure injection
7.8 Results page: QoE breakdown and logs
7.9 Experiment 6: real socket baseline
7.10 Discussion

8 Testing and Verification

9 Limitations and Threats to Validity

10 Conclusion and Future Work

References

Appendix Running the project

1 Introduction

1.1 Background and motivation

Video is the dominant consumer of Internet capacity, and viewers judge a streaming service by a handful of observable events: how long the picture takes to start, whether it freezes, and how sharp it looks. Behind those events sits a long causal chain. The access network offers a bandwidth that changes from second to second, adds delay and jitter, and occasionally drops packets. The transport protocol turns that behaviour into delivery times for each chunk of video. The player's buffer and its adaptive bitrate (ABR) algorithm turn delivery times into a choice of quality and, when the buffer runs dry, into a stall [1][2].

Two transport protocols sit at the heart of this chain. The Transmission Control Protocol (TCP) offers reliable, ordered, congestion-controlled delivery [3], at the price of connection set-up, retransmission delay and head-of-line (HOL) blocking. The User Datagram Protocol (UDP) offers independent, unreliable datagrams with almost no overhead [4], but leaves reliability, ordering and rate control to the application. QUIC, built on UDP, re-introduces reliability and congestion control while giving each stream independent ordering [5][6]. Which one gives viewers a better experience is not a matter of opinion: it depends on the loss rate, the round-trip time (RTT), the available bandwidth, the recovery policy and the ABR algorithm.

1.2 Case study: how Netflix streams video

The case study chosen for this project is "Transport Layer in Practice – How Netflix Streams Videos Using TCP and UDP". Publicly documented accounts describe Netflix's Open Connect content-delivery network as a fleet of caching appliances placed inside or near Internet service provider (ISP) networks [7]. A control plane authenticates the client and steers it to an appliance; the data plane then serves video segments over HTTP(S) on TCP, while the client itself selects the bitrate of each segment using ABR logic. UDP-based transports appear in other roles, notably QUIC/HTTP-3 deployments and real-time traffic. This project does not claim to reproduce any proprietary Netflix system; it uses these public descriptions as the motivating scenario and builds an open, inspectable simulator around them.

1.3 Problem statement

Textbooks state that TCP is reliable but can suffer head-of-line blocking, and that UDP is fast but unreliable. Such statements are qualitative and often misleading when applied to video, where a segment must be complete before it can be played, where buffering hides some delay, and where the ABR algorithm reacts to whatever the transport delivers. The problem addressed here is therefore:

How do transport-layer behaviour (TCP-like, UDP-like and QUIC-like delivery) and ABR adaptation jointly determine the rebuffering, quality and overall Quality of Experience (QoE) of segmented video streaming under changing bandwidth, latency, jitter and packet loss, and why?

1.4 Objectives

1. Build an interactive, reproducible, seeded simulator that links network conditions, transport behaviour, segmented delivery, playback buffering, ABR decisions and QoE.
2. Model TCP-like, UDP-like and (as an extension) QUIC-like transports at a transparent, teachable level, stating clearly what is and is not simulated.
3. Implement throughput-based and buffer-based ABR and log the rule and reason behind every bitrate decision.
4. Define a documented, configurable QoE score and report each component next to the total.
5. Compare transports and ABR strategies under identical bandwidth traces, loss draws and seeds, so that only the factor under study changes.
6. Support conclusions with statistics: 30 independent seeds, 95% confidence intervals, a comparison with the Mathis throughput approximation, and sensitivity analysis.
7. Provide an interactive dashboard, a command-line interface, automated tests and exportable results.

1.5 Scope and contributions

The project is an educational simulation. It models one client, one bottleneck path and one video session at a time; it does not model cross traffic, CDN server selection, DRM, audio tracks or real codecs. Its contributions are:

- **A transport-to-QoE pipeline** whose causal chain is observable end to end, with event logs for every handshake, retransmission, timeout, bitrate decision and stall.
- **A fair comparison harness**: the bandwidth trace depends only on the seed, and the loss draw for the first transmission of every packet is keyed by (seed, segment, packet), so all transports face the same network.
- **Explainable ABR**: every decision stores its inputs, the rule that fired and a human-readable reason.
- **Statistical rigour**: repeated seeds, Student-t confidence intervals, explicit exclusion reporting and a regime-aware validation against the Mathis model.
- **Reproducibility**: the same configuration and seed yield identical output; 98 automated tests cover the engine, statistics and API.

1.6 Organisation of the report

Section 2 reviews the background and related work. Section 3 presents the system architecture and Section 4 the models and equations. Section 5 describes the implementation, Section 6 the experimental methodology and Section 7 the results with screenshots from the running application. Section 8 covers testing, Section 9 limitations and threats to validity, and Section 10 concludes with future work.

2 Background and Literature Review

2.1 The transport layer

2.1.1 Transmission Control Protocol

TCP provides a reliable, in-order byte stream. Reliability comes from sequence numbers, cumulative acknowledgements and retransmission; rate control comes from a congestion window (cwnd) that grows in slow start and congestion avoidance and is reduced on loss [8][9]. Lost segments are recovered either by fast retransmit, triggered by three duplicate acknowledgements, or, when too few packets follow the loss, by a retransmission timeout (RTO) whose lower bound is 200 ms in common implementations and whose value doubles on repeated loss [10]. Later congestion controllers such as CUBIC and BBR replace the AIMD rule with different window or rate models [11], but the loss-recovery and ordering behaviour described here is common to them. Because delivery to the application is strictly in order, packets that arrive behind a missing one wait in the receive buffer until the hole is repaired. This is **head-of-line blocking**.

2.1.2 User Datagram Protocol

UDP adds only port numbers and a checksum to IP [4]. There is no connection set-up, acknowledgement, retransmission, ordering or congestion control. Datagrams can be lost, duplicated or reordered, and a UDP sender that ignores the network will overrun any bottleneck. Applications that need completeness over UDP must implement their own recovery: negative acknowledgements and resend (ARQ), forward error correction (FEC), or tolerance of gaps.

2.1.3 QUIC

QUIC runs over UDP and integrates the cryptographic handshake with transport set-up, so that a new connection needs one round trip instead of the two typically needed for TCP plus TLS 1.3 [5]. Streams are ordered independently, so a loss on one stream does not delay another; loss detection and congestion control are specified separately [12]. Large-scale deployment results have been reported by Langley et al. [6].

2.2 Throughput under loss: the Mathis model

Mathis et al. derived that the steady-state throughput of a long-lived AIMD (Reno-like) TCP flow under random loss probability p is bounded by [13]:

$$\text{Throughput} \approx (\text{MSS} / \text{RTT}) \cdot C / \sqrt{p},\ C \approx 1.22$$

Padhye et al. extended the analysis to include timeouts, which dominate at higher loss rates [14]. The Mathis approximation assumes periodic loss, no timeouts and a window that is limited by loss rather than by capacity or the receiver; this project uses it as a sanity check in the regime where those assumptions hold and flags points outside it.

2.3 HTTP adaptive streaming and ABR

In HTTP adaptive streaming, content is encoded at several bitrates (the "ladder") and cut into segments of a few seconds; the client downloads one segment at a time and chooses the bitrate of the next [1][15]. Published ABR families include:

- **Throughput-based** algorithms such as FESTIVE estimate bandwidth from recent downloads and pick the highest sustainable rung [16].
- **Buffer-based** algorithms such as BBA map buffer occupancy to a bitrate, trading throughput information for robustness [17].
- **Control-theoretic / learning** approaches combine both signals (model predictive control [18]) or learn policies from experience [19].

This project implements the first two families, because they differ in precisely what they observe: throughput-based ABR sees the transport, while buffer-based ABR sees only its consequence.

2.4 Quality of Experience

Large-scale studies show that start-up delay, rebuffering and average bitrate measurably change viewer engagement [20][21], and surveys catalogue the QoE factors of HTTP adaptive streaming [2]. Linear QoE models of the form "quality minus rebuffering minus switching minus start-up penalties" are widely used in ABR research [18]. The score in this project follows that structure with documented, adjustable weights and is explicitly not an industry-standard measurement.

2.5 Related tools and the gap addressed

Table 1. Related tools and what this project adds

Tool / work	Strength	Limitation for this study
ns-3 [22]	Packet-level protocol fidelity, real TCP/UDP stacks	Heavy setup; no built-in video buffer, ABR or QoE model
Mahimahi [23]	Record-and-replay of real HTTP over emulated links	Needs real servers and root privileges; not a self-contained teaching tool
Pensieve and ABR papers [19][17][18]	State-of-the-art ABR evaluation	Compare ABR algorithms, not the effect of the transport protocol
TCPlay (this project)	Transport → buffer → ABR → QoE chain, identical conditions, statistics, dashboard	Simplified protocol models (see Section 9)

The gap is a lightweight, transparent tool that isolates *transport behaviour* as the only varying factor and explains the outcome through logged events, rather than treating the protocol as a black box.

3 System Design and Architecture

3.1 Requirements

Table 2. Functional and non-functional requirements

ID	Requirement	How it is met
F1	Configurable bandwidth, RTT, jitter, loss, traces, seeds	NetworkConfig, seeded trace generator, impairment windows
F2	TCP-like, UDP-like and QUIC-like delivery	tcp_model, udp_model, quic_model
F3	Segmented video with ladder, buffer, startup and rebuffering	experiment, buffer
F4	Throughput- and buffer-based ABR with decision logs	abr
F5	Transparent QoE with visible components	qoe
F6	Controlled comparisons, repeated trials, CIs, Mathis check, sensitivity	lab, stats
F7	Dashboard, CLI and export	React app, app.cli, export
N1	Reproducibility: same config + seed ⇒ same output	Keyed random streams; tested
N2	Engine independent of the UI	Pure-Python package; API/CLI/tests call it directly
N3	Honest reporting: all values measured from runs	No stored or canned numbers; exports carry config + seed

3.2 Architecture

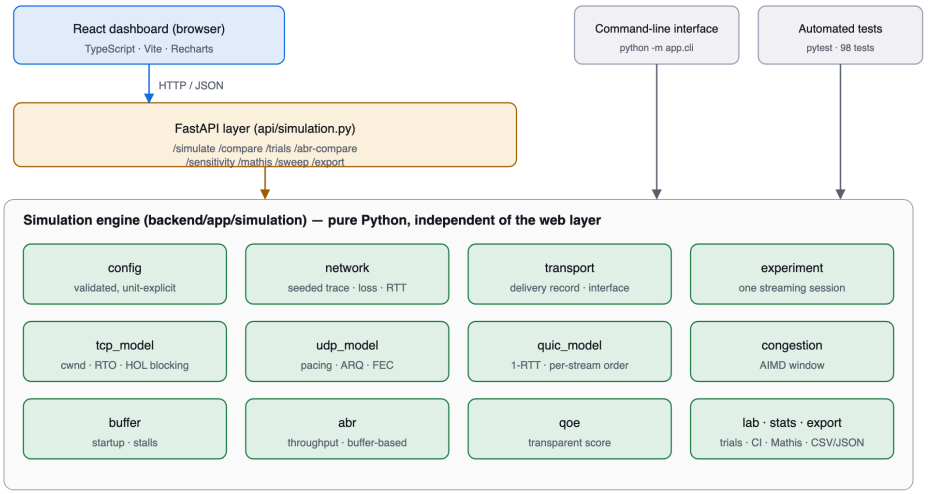


Figure 1. TCPlay software architecture. The dashboard, CLI and tests all drive the same engine.

The system has three layers. The presentation layer is a React and TypeScript single-page application using Vite and Recharts. The service layer is a FastAPI application that validates requests and exposes the engine over JSON. The engine itself, under backend/app/simulation, is plain Python with NumPy for vectorised random draws; it has no dependency on the web framework, which is why the same code serves the dashboard, the command-line tool and the 98 unit and integration tests.

3.3 Conceptual pipeline

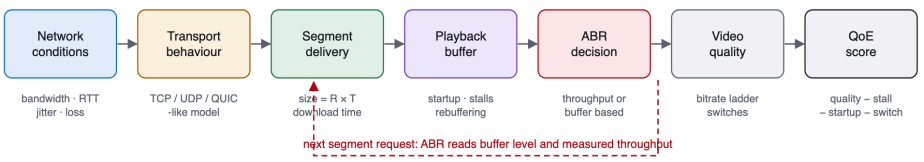


Figure 2. The causal chain simulated for every video segment, with the ABR feedback loop.

The measured goodput of each segment is the signal through which the transport influences the next ABR decision, which is why transport and ABR cannot be studied independently.

3.4 Module descriptions

Table 3. Engine modules and responsibilities

Module	Responsibility
config.py	Unit-explicit dataclasses, validation, from_dict (rejects unknown keys), network presets
network.py	Seeded bandwidth traces (constant, random walk, time-stepped), impairment windows, loss/RTT/jitter, keyed random draws
transport.py	SegmentDelivery record and abstract transport interface
congestion.py	Reno-style AIMD window: slow start, avoidance, fast loss, timeout, idle restart
tcp_model.py	TCP-like transport: handshake, cwnd rounds, fast retransmit vs RTO, HOL blocking
udp_model.py	UDP-like transport: pacing, bottleneck queue, jitter and reordering, recovery policy (none / ARQ / FEC)
quic_model.py	QUIC-like extension: 1-RTT set-up, per-stream ordering, fast loss timer
buffer.py	Playback buffer: growth, depletion, startup, stalls; never negative
abr.py	Throughput-based and BBA-style buffer-based ABR with decision logging
qoe.py	Transparent, configurable QoE score
experiment.py	One seeded streaming session producing segments, events, timeline, metrics
lab.py / stats.py / export.py	Comparisons, trials, sweeps, sensitivity, Mathis; mean, std, t-interval; JSON/CSV

3.5 Units and configuration

Every configuration field carries its unit in its name, and unit conversions are tested explicitly (Section 8).

Table 4. Unit conventions

Quantity	Unit in configuration	Internal / conversion
Bandwidth, bitrate	Mbps (10^6 bit/s)	bit/s = Mbps × 10^6
Segment size	megabits = R × T	bytes = Mbit × 10^6 / 8
RTT, jitter	milliseconds	seconds = ms / 1000
Loss	probability per packet (0.01 = 1%)	compared with a uniform draw
Packet payload (MSS)	bytes (default 1460)	packets = ⌈bytes / MSS⌉
Time, buffer	seconds	buffer measured in seconds of video

4 Simulation Models and Methodology

4.1 Network model

A bandwidth trace is a piecewise-constant function of time. Three generators are provided: a constant rate; a seeded, mean-reverting random walk in log space,

$$\log b_{t+1} = \log b_t + \theta\,(\log \mu - \log b_t) + N(0, \sigma^2),\ \theta = 0.15,\ \text{clipped to } [b_min, b_max];$$

and a user-supplied list of (time, Mbps) steps. Failure-injection windows can scale bandwidth, add loss or add RTT for an interval. Packet loss is an independent Bernoulli process with the configured probability, plus tail-drop loss whenever a burst exceeds the bandwidth-delay product plus the bottleneck queue (100 packets by default). Jitter is the standard deviation of per-packet one-way delay variation.

Controlled randomness. Random draws are keyed by (seed, stream, segment, attempt): the bandwidth trace depends only on the seed, and the loss draw for the first transmission of packet i of segment s is the same for TCP-like, UDP-like and QUIC-like delivery. Retransmissions use independent draws. This is what makes a transport comparison fair and the whole simulator deterministic.

4.2 TCP-like transport

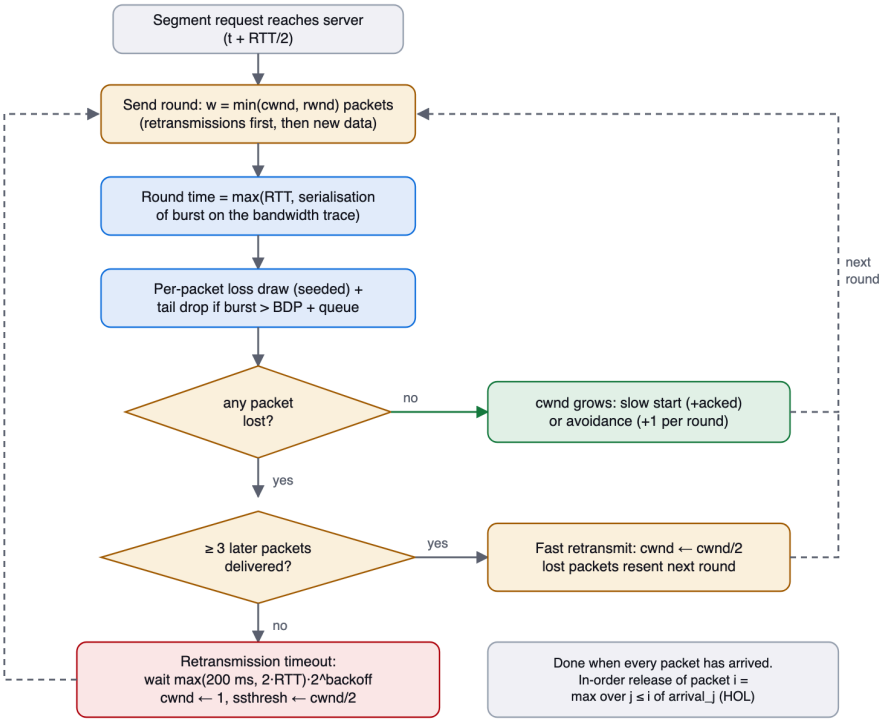


Figure 3. Flowchart of one TCP-like segment transfer (round-based model).

Transfers proceed in rounds. A round sends w = min(cwnd, rwnd) packets, retransmissions first. Its duration is the larger of the RTT sample and the serialisation time of the burst on the bandwidth trace, so throughput is approximately min(cwnd·MSS/RTT, bandwidth). Losses are classified as follows: a lost packet followed by at least three delivered packets in the same round is a *fast retransmit* (window halved, repair in the next round); otherwise a *retransmission timeout* fires after max(200 ms, 2·RTT) with exponential backoff, and the window collapses to one packet.

$$\text{RTO} = \max(\text{min_RTO}, 2 \cdot \text{RTT}) \cdot 2\text{^backoff},\ \text{cwnd} \leftarrow 1,\ \text{ssthresh} \leftarrow \max(\text{cwnd}/2, 2)$$

Head-of-line blocking is measured exactly from the per-packet arrival times arrival_j: the in-order release time of packet i is the maximum arrival time over all packets up to i, and the blocking time is the sum of (release − arrival) over packets, in packet·seconds.

$$\text{release}_i = \max_{j \le i} \text{arrival}_j, \quad \text{HOL} = \sum_i (\text{release}_i - \text{arrival}_i)$$

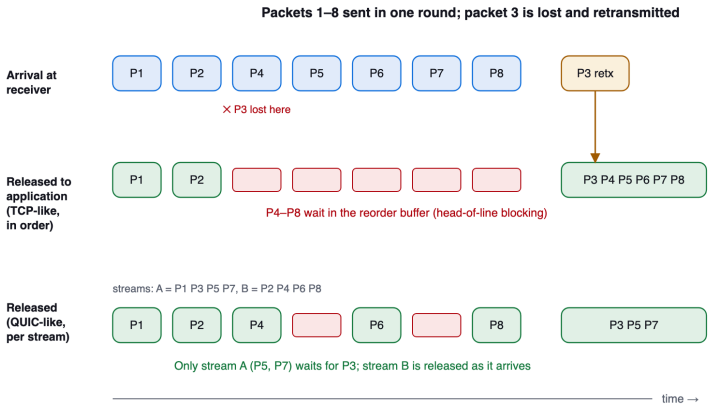


Figure 4. Head-of-line blocking. Packet 3 is lost; in TCP-like delivery packets 4–8 wait for its retransmission, whereas with two independent QUIC-like streams only the stream that lost the packet waits.

4.3 UDP-like transport

The UDP-like sender has no handshake and no congestion control. It paces datagrams at *pacing factor* × *selected bitrate* (3× by default, a sender policy that also lets throughput-based ABR discover spare capacity). If that rate exceeds the bottleneck, the queue overflows and datagrams are dropped. Each datagram receives its own one-way delay, so jitter produces reordering; datagrams arriving after the receiver's gap timer are counted as late. Because UDP never retransmits, completeness is an *application* decision:

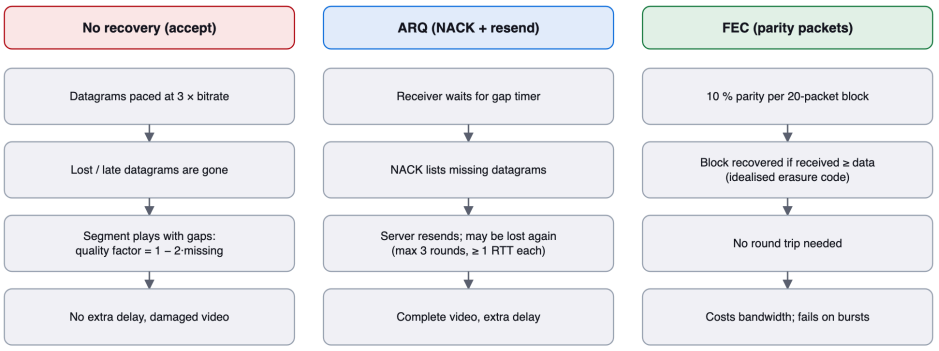


Figure 5. The three application-level recovery policies implemented over the UDP-like transport.

A segment that remains incomplete is either played with a concealment penalty (quality factor $q = \max(0, 1 - 2 \cdot \text{missing fraction})$) or discarded. This is why the answer to "is UDP better?" is not a property of UDP alone.

4.4 QUIC-like transport

The QUIC-like mode reuses the reliable-delivery engine and changes only three properties that distinguish QUIC in this teaching model: a one-round-trip set-up, independent ordering of four streams per segment, and a 25 ms floor on the loss timer instead of 200 ms. It is not an implementation of QUIC: there are no packet-number spaces, ACK ranges, 0-RTT or flow-control credit, and loss detection is not RFC 9002.

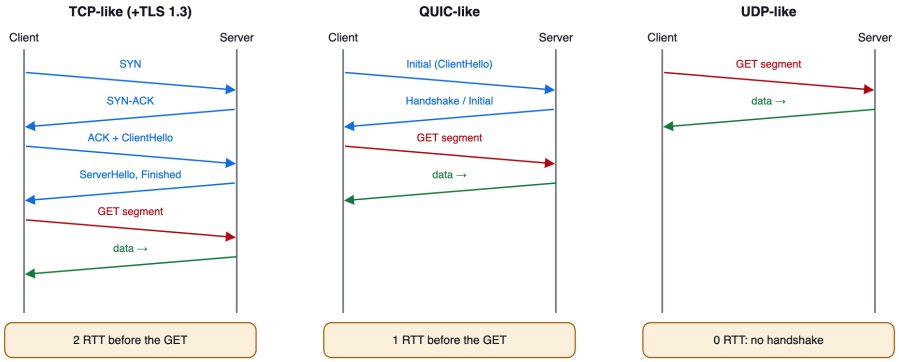


Figure 6. Connection set-up cost before the first segment request in the three transport models.

4.5 Video, buffer and playback

A video of duration D is cut into segments of T seconds (default 120 s and 4 s). The ladder is 240p 0.35, 360p 0.75, 480p 1.5, 720p 3 and 1080p 6 Mbps. A segment encoded at R Mbps has R × T megabits. The playback buffer holds seconds of downloaded video: it increases by T when a download completes and decreases at one second per second while playing. It cannot become negative; reaching zero starts a stall unless all content has been played. Playback starts when 8 s are buffered and resumes after a stall at 4 s; the client idles when buffer + segment would exceed 30 s.

4.6 Adaptive bitrate algorithms

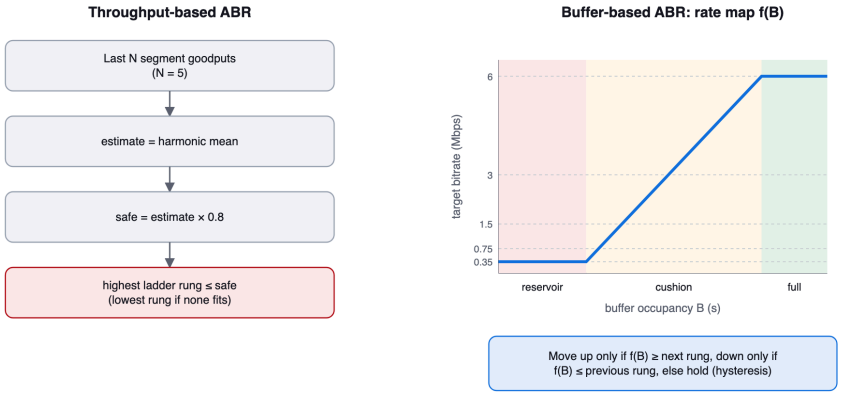


Figure 7. Decision logic of the two ABR strategies.

$$\text{Throughput: est} = \text{harmonic_mean(last N goodputs)};\ \text{safe} = \text{est} \times \text{safety};\ \text{rung} = \max\{R \le \text{safe}\}$$

The harmonic mean is dominated by slow samples, so a single fast segment does not cause an over-optimistic jump. The buffer-based strategy follows the BBA family [17]: it maps buffer level to a target rate between a reservoir (8 s) and reservoir + cushion (24 s), but moves only when the map crosses the adjacent rung, which suppresses oscillation. The first segment always uses the configured initial rung. Every decision records the buffer level, estimate, rule name and a textual reason.

4.7 QoE model

Let $U(R) = \ln(1 + R/R_{min}) / \ln(1 + R_{max}/R_{min})$ be a concave utility of bitrate, and $q_k \in [0, 1]$ the delivered-quality factor of segment k (1 unless a UDP segment was damaged or dropped). The score is:

$$\text{quality} = 100 \cdot \text{mean}_k\left(U(R_k) \cdot q_k \right)$$
$$\text{rebuffer} = w_r \cdot 100 \cdot \text{stall} / (\text{content} + \text{stall}) + w_e \cdot n_{stalls}$$
$$\text{startup} = \min(\text{cap}, w_s \cdot \text{startup_delay})$$
$$\text{switching} = w_{sw} \cdot 100 \cdot \text{mean}_k \left| U(R_{k+1}) - U(R_k) \right|$$
$$\text{QoE} = \text{clamp}(\text{quality} - \text{rebuffer} - \text{startup} - \text{switching}, 0, 100)$$

Table 5. Default QoE weights

Symbol	Meaning	Default
w_r	Points per 1% of session time stalled	4
w_e	Points per stall event	2
w_s	Points per second of start-up delay (cap 20)	2
w_sw	Weight on mean utility change between segments	0.5

The weights are judgement calls and are editable in the dashboard; they are not calibrated against subjective data, so absolute scores matter less than comparisons under the same weights.

4.8 Session algorithm

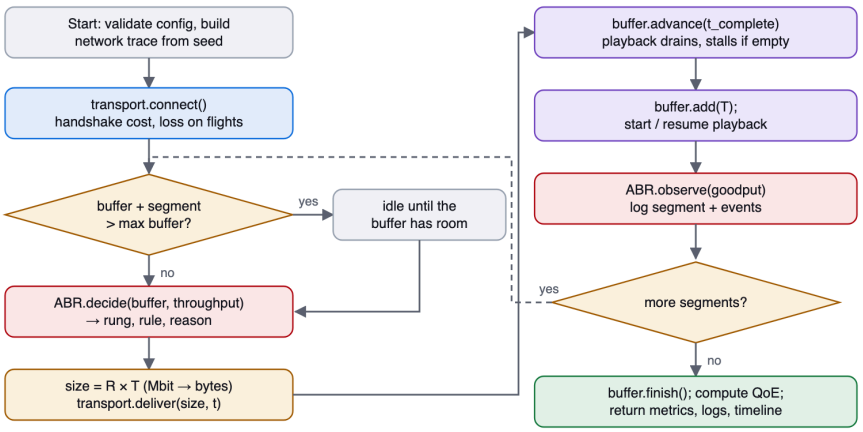


Figure 8. Flowchart of one simulated streaming session.

4.9 What is simulated and what is not

Table 6. Modelling assumptions and omissions

Aspect	Simulated	Not simulated
TCP	Handshake, cwnd, fast retransmit, RTO + backoff, HOL, slow start after idle	ACK clocking, delayed ACK, SACK, RACK/TLP, CUBIC/BBR, pacing, header overhead
UDP	Pacing, queue drops, jitter, reordering, ARQ, idealised FEC	RTP/RTCP, real codecs' error propagation, real FEC codes, NAT
QUIC	1-RTT set-up, per-stream order, fast timer	Packet-number spaces, 0-RTT, RFC 9002 detection
Network	Seeded trace, i.i.d. loss, queue tail drop, jitter, fault windows	Cross traffic, bursty loss, correlated jitter, multi-hop routing
Video	Constant-bitrate ladder, segment buffer, startup, stalls	Variable bitrate, audio, subtitles, DRM, decode time

5 Implementation

5.1 Technology stack

The engine is written in Python 3.12 with NumPy; the service layer uses FastAPI and Pydantic; the dashboard uses React, TypeScript, Vite, Tailwind CSS and Recharts. Backend tests use pytest and the frontend is type-checked with the TypeScript compiler.

5.2 Backend and API

The API is a thin wrapper: each endpoint validates its input (configuration errors become HTTP 422 with the offending field named), calls the engine and returns the result unchanged.

The API exposes /simulate (one session), /compare (same-seed transport comparison), /trials (N seeds with confidence intervals), /abr-compare, /sensitivity, /mathis, /sweep and /export (JSON/CSV).

5.3 Command-line interface

The same functions are available without a web server, which is how the committed result files were produced:

```
python -m app.cli run --preset mobile_4g --protocol tcp --seed 7
python -m app.cli compare-trials --n 30 --out ../results/transports.json
python -m app.cli mathis --n 10 --csv ../results/mathis.csv
python -m app.cli abr --preset sudden_drop --n 30
python -m app.cli sensitivity --param abr.safety_factor --values 0.4,0.6,0.8,1.0 --n
20
```

5.4 Dashboard

The dashboard has nine pages: Home, Experiment Lab (configuration, single session, repeated trials), TCP vs UDP, ABR Lab, Live Simulation (replay with failure injection), Results (QoE breakdown, segment and event logs, export), What-If Analysis, Model Validation and Experiment History.

5.5 Reproducibility and export

Every result contains its full configuration and seed. JSON export preserves the whole structure; CSV export flattens segments, events, timelines and per-seed metrics. Re-running a stored configuration reproduces the stored result exactly, which a unit test asserts.

6 Experimental Design

6.1 Research questions

1. RQ1. Under identical conditions, how do TCP-like, UDP-like (with different recovery policies) and QUIC-like delivery differ in QoE, and why?
2. RQ2. Does the TCP-like model reproduce the expected $1/(RTT \cdot \sqrt{p})$ throughput law in the regime where it should?
3. RQ3. How do throughput-based and buffer-based ABR behave under the same traces?
4. RQ4. Which parameters (safety factor, segment duration, loss, …) most affect QoE?

6.2 Default scenario

Table 7. Default experimental configuration

Parameter	Value
Mean bandwidth / profile	8 Mbps, seeded random walk (σ = 0.25 per 2 s step), clipped to [0.3, 24] Mbps
RTT / jitter / loss	50 ms / 5 ms / 0.5 %
Bottleneck queue	100 packets
Video	120 s, 4 s segments (30 segments), ladder 0.35–6 Mbps
Buffer	Startup 8 s, resume 4 s, maximum 30 s
ABR	Throughput-based, safety 0.8, harmonic mean of last 5 samples
TCP-like	MSS 1460 B, initial cwnd 10, receive window 1000 packets, min RTO 200 ms, 2-RTT set-up
UDP-like	Pacing 3× bitrate, ARQ up to 3 rounds, FEC 10 % per 20-packet block
Seeds	1…30 (independent traces and loss draws)

6.3 Experiments

Table 8. Experiment plan

#	Experiment	Independent variable	Main outputs
1	Transport comparison	Transport / UDP recovery policy	QoE, bitrate, HOL blocking, incomplete segments
2	Loss × RTT sweep and Mathis check	$p \in \{0.1, 0.3, 1, 3\}$%, RTT $\in \{20 \ldots 200\}$ ms	Throughput vs $(MSS/RTT) \cdot 1.22/\sqrt{p}$, errors
3	Repeated trials	Seed (30)	Mean, std dev, 95% CI, n, exclusions
4	ABR comparison	ABR strategy, trace	QoE, bitrate, stalls, switches
5	Sensitivity analysis	Safety factor, segment length, loss	QoE and bitrate vs parameter
6	Real socket baseline (optional)	TCP vs UDP on loopback	Connect time, goodput, UDP loss

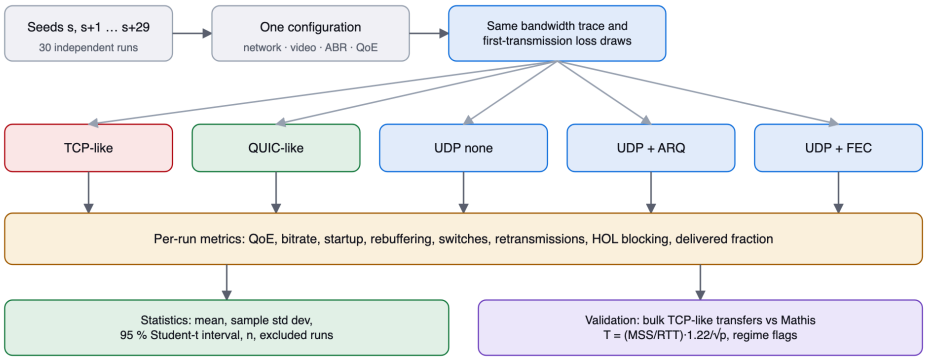


Figure 9. Experimental methodology: identical seeds and traces for every variant, then statistics.

6.4 Statistical method

For each metric over n seeds the mean $\bar{x}$, the sample standard deviation s ($n - 1$ denominator) and a two-sided 95% confidence interval are reported using Student's t-distribution [24]:

$$\mathrm{CI}_{95} = \bar{x} \pm t(0.975, n - 1) \cdot s / \sqrt{n} \; (t = 2.045 \text{ for } n = 30)$$

The interval assumes independent observations and roughly normal means. With 30 seeds the central limit theorem makes this reasonable, but metrics that are zero for most seeds, such as rebuffering time, violate the assumption and are flagged. Runs that fail are excluded *and listed with the reason*; in all experiments reported here no run was excluded.

6.5 Validation against theory

The TCP-like model is validated by bulk transfers on an uncongested 1 Gbit/s path so that loss, not capacity, limits the window. Throughput is compared with the Mathis approximation for $p > 0$ only (the formula diverges at $p = 0$). A point is flagged as outside the formula's regime when the predicted window is below 15 packets, p exceeds 2%, or capacity or the receive window is binding.

7 Results and Discussion

7.1 Application walkthrough with sample inputs

This section shows the running application with concrete inputs, captured from the dashboard with seed 1 and the default scenario described in Section 6.2. The two figures below show the configuration panels of the Experiment Lab: network conditions (bandwidth 8 Mbps random walk, RTT 50 ms, jitter 5 ms, loss 0.5%) and the transport model.

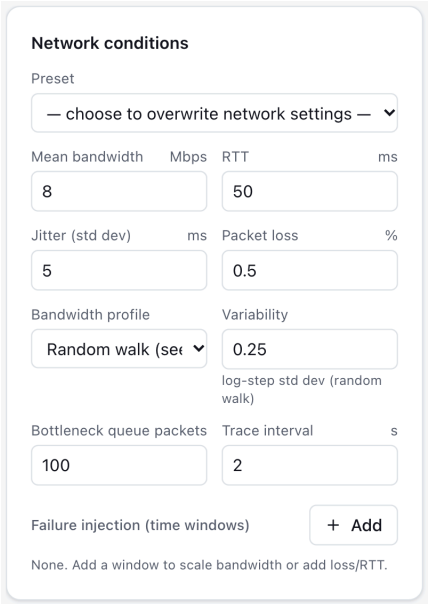


Figure 10. Experiment Lab – network conditions panel with the default inputs.

Pressing "Run session" executes one seeded streaming session and renders the metrics and time series directly from the engine output:

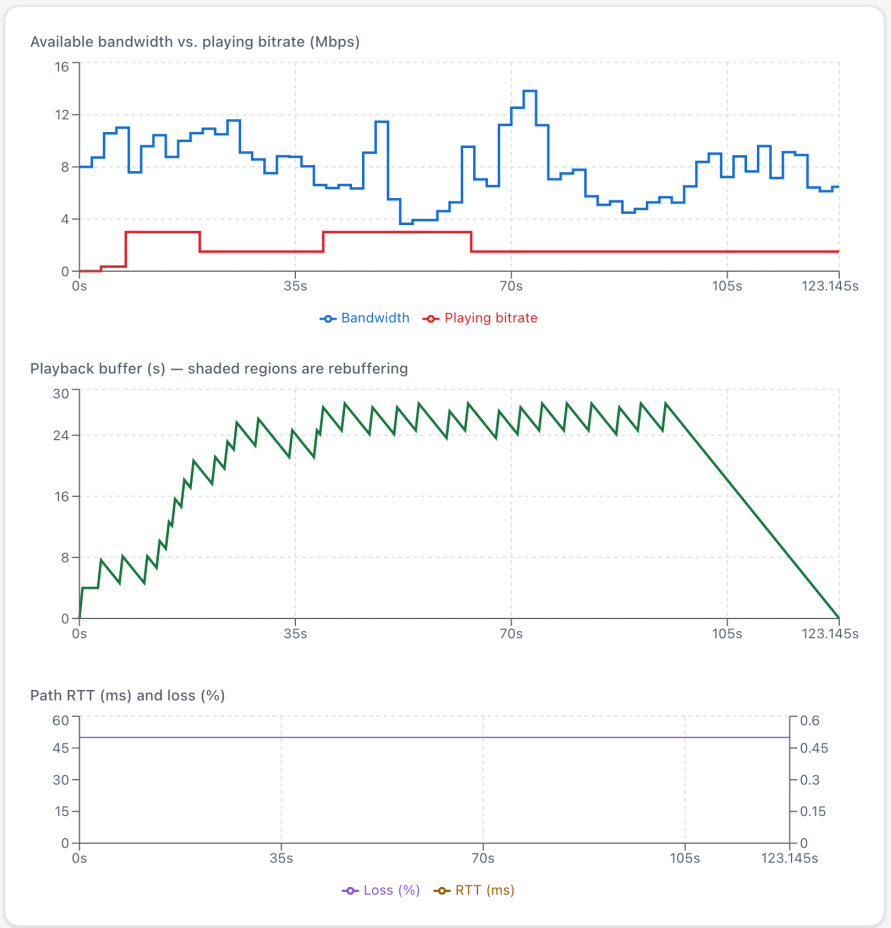


Figure 11. Experiment Lab – available bandwidth versus playing bitrate, playback buffer and path RTT/loss for one TCP-like session (seed 1).

Repeated trials · TCP-like · throughput ABR

Metric	Mean	Std dev	95% CI low	95% CI high	Min	Max	n
QoE score (0–100)	**54.49**	3.52	53.18	55.81	47.30	63.95	30
Average bitrate (Mbps)	**1.899**	0.249	1.806	1.992	1.437	2.462	30
Startup delay (s)	**2.88**	0.65	2.64	3.13	1.70	4.21	30
Rebuffering time (s)	**0.00**	0.00	0.00	0.00	0.00	0.00	30
Rebuffer events	**0.00**	0.00	0.00	0.00	0.00	0.00	30
Bitrate switches	**4.33**	1.83	3.65	5.01	2.00	8.00	30
Mean segment goodput (Mbps)	**3.598**	0.283	3.493	3.704	2.906	4.119	30
Mean segment download (s)	**2.249**	0.165	2.188	2.311	1.880	2.563	30
Retransmissions	**109.7**	16.7	103.5	115.9	85.0	145.0	30
Timeouts	**19.4**	4.4	17.8	21.0	10.0	30.0	30
HOL blocking (packet·s)	**34.02**	5.15	32.10	35.94	25.18	47.24	30
Incomplete segments	**0.00**	0.00	0.00	0.00	0.00	0.00	30
Mean delivered fraction	**1.0000**	0.0000	1.0000	1.0000	1.0000	1.0000	30

30 of 30 runs completed (seeds 1–30). Excluded runs: none. CI method: Student t, 95% two-sided, df = n-1 (assumes independent, roughly normal means).

Figure 12. "Run 30 seeds" – mean, standard deviation, 95% confidence interval and n for each metric.

7.2 Experiment 1: TCP-like vs UDP-like vs QUIC-like

The same seed produces the same bandwidth trace and the same first-transmission loss draws for every variant. The figures below show one such comparison in the dashboard (seed 1); the table that follows reports 30 seeds.

Metric	TCP	UDP (NONE)	UDP (ARQ)	QUIC
QoE score	54.2	39.5	59.7	58.6
Avg bitrate (Mbps)	1.91	2.49	2.39	2.16
Startup delay (s)	3.14	2.79	2.83	3.00
Rebuffering (s)	0.00	0.00	0.00	0.00
Rebuffer events	0	0	0	0
Bitrate switches	4	5	5	2
Goodput (Mbps)	3.79	5.26	5.16	3.83
Segment download (s)	2.18	1.49	1.85	2.35
Retransmissions	128	0	4284	138
Timeouts	18	0	0	21
HOL blocking (pkt·s)	33.3	0.0	0.0	6.5
Reordered packets	0	11462	11490	0
Incomplete segments	0	25	0	0
Delivered fraction	1.0000	0.8672	1.0000	1.0000

Single run, seed 1. Rebuffering is zero for many seeds, so single runs can mislead — use trials.

Figure 13. TCP vs UDP page – single-seed comparison table (seed 1).

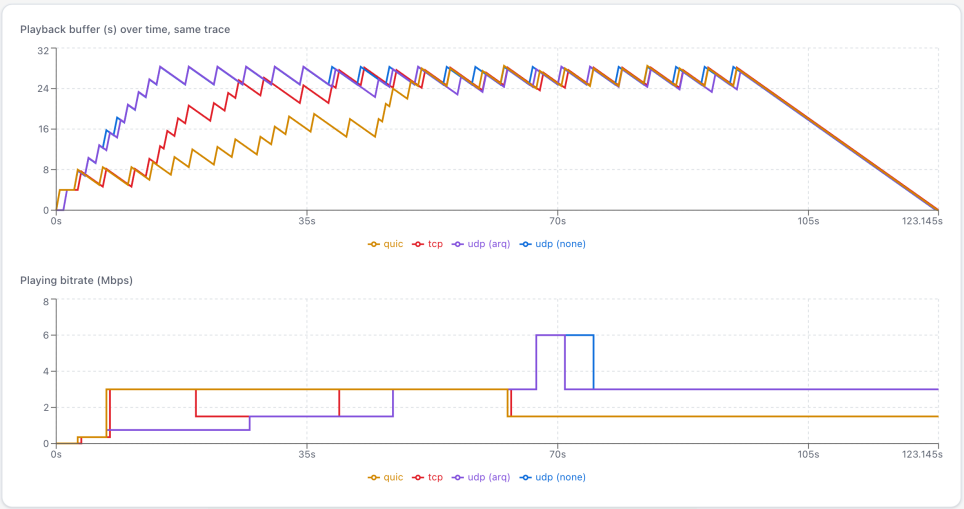


Figure 14. playback buffer for each transport on the identical trace; UDP + ARQ fills the buffer fastest, QUIC-like slowest.

Table 9. Experiment 1: transport comparison, mean ± 95% CI over 30 seeds (default scenario)

Variant	QoE	Avg bitrate (Mbps)	Startup (s)	Goodput (Mbps)	HOL (pkt·s)	Incomplete segs
TCP-like	54.5 ± 1.3	1.90 ± 0.09	2.88	3.60	34.0	0.0
QUIC-like	55.9 ± 1.5	2.00 ± 0.10	2.79	3.67	6.5	0.0
UDP-like, no recovery	38.1 ± 1.5	2.70 ± 0.18	2.78	5.55	0	27.7
UDP-like, ARQ	60.5 ± 1.6	2.59 ± 0.18	2.84	5.31	0	0.0
UDP-like, FEC (10 %)	41.9 ± 2.2	2.41 ± 0.18	2.99	4.94	0	11.5

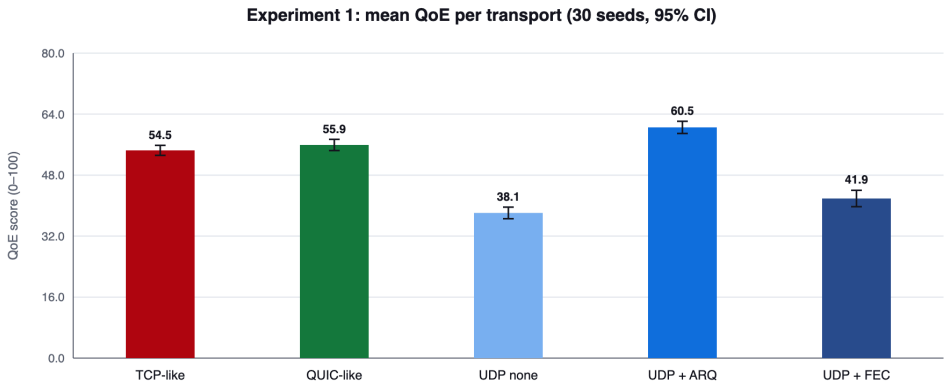


Figure 15. Mean QoE per transport with 95% confidence intervals (30 seeds).

Observations.

- UDP-like delivery without recovery reaches the *highest* average bitrate (2.70 Mbps against 1.90 Mbps for TCP-like), because it neither waits for retransmissions nor slows down. But 27.7 of 30 segments arrive damaged (delivered fraction 0.858), so its QoE of 38.1 is the lowest.
- Adding ARQ repairs the missing datagrams at the cost of extra round trips and 5499 resent datagrams per session; every segment is complete and the QoE of 60.5 is the highest of all variants, 6.0 points above TCP-like.
- FEC with 10 % parity recovers only part of the damage (11.5 incomplete segments): the 0.5 % random loss is easily covered, but a likely reason for the remainder is overload loss at the bottleneck queue (the sender does not back off), whose bursts exceed what a 2-parity block can repair.
- TCP-like and QUIC-like deliver every segment intact; QUIC-like is 1.4 QoE points higher, but the confidence intervals overlap, so the difference is not established at 30 seeds.
- No variant rebuffered in this scenario: the 30 s buffer and a conservative ABR absorb the delivery-time differences. The transport shows up in quality, not in stalls.

7.2.1 Head-of-line blocking and incomplete delivery

Packets waited an average of 34.0 packet·seconds in the TCP-like receive buffer behind lost packets. With four independent streams the QUIC-like model cut this to 6.5 (a factor of about 5.2), because only packets in the stream that lost a packet wait. Retransmissions are almost identical (110 vs 115): the loss recovery is the same; what changes is *who has to wait*. The TCP-like model also experienced 19 timeouts per session, mainly tail losses at the end of a segment that cannot trigger fast retransmit; each costs at least 200 ms.

7.3 Experiment 2: TCP-like throughput vs the Mathis approximation

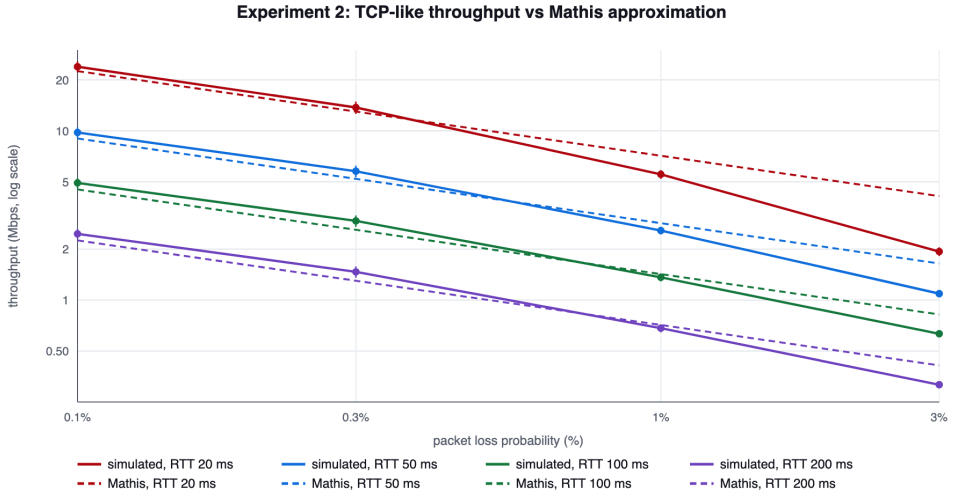


Figure 16. Simulated bulk-transfer throughput (mean ± 95% CI over 10 seeds) against the Mathis approximation for four RTTs.

Table 10. Experiment 2: simulated vs Mathis throughput (10 seeds per point)

RTT (ms)	Loss p	Simulated (Mbps)	Mathis (Mbps)	Rel. error	Window (pkts)	Assumptions
20	0.001	23.94	22.53	+6 %	39	hold
20	0.003	13.73	13.01	+6 %	22	hold
20	0.01	5.53	7.12	-22 %	12	violated
20	0.03	1.94	4.11	-53 %	7	violated
50	0.001	9.78	9.01	+8 %	39	hold
50	0.003	5.77	5.20	+11 %	22	hold
50	0.01	2.58	2.85	-10 %	12	violated
50	0.03	1.09	1.65	-34 %	7	violated
100	0.001	4.92	4.51	+9 %	39	hold
100	0.003	2.94	2.60	+13 %	22	hold
100	0.01	1.36	1.42	-4 %	12	violated
100	0.03	0.63	0.82	-23 %	7	violated
200	0.001	2.46	2.25	+9 %	39	hold
200	0.003	1.47	1.30	+13 %	22	hold
200	0.01	0.68	0.71	-4 %	12	violated
200	0.03	0.32	0.41	-23 %	7	violated

Where the formula's assumptions hold (predicted window of at least 15 packets and $p \leq 0.3$ %, eight points), the simulated throughput is 6–13 % above the bound and the mean absolute relative error is 9.4 %. Throughput halves when RTT doubles and falls as $1/\sqrt{p}$, which confirms the model's scaling. At $p = 1$ % the window is only about 12 packets; tail losses and timeouts become frequent, and the simulation falls 4–22 % below the bound; at $p = 3$ % (about 7 packets) it is 23–53 % below. This is the expected direction: Mathis ignores timeouts, which Padhye et al. showed dominate at high loss [14]. The comparison therefore supports using the model in the moderate-loss regime and documents where it should not be trusted.

7.4 Experiment 3: repeated trials

Table 11. Experiment 3: default configuration, TCP-like, throughput ABR, seeds 1–30

Metric	Mean	Std dev	95% CI low	95% CI high	n
QoE score	54.49	3.52	53.18	55.81	30
Average bitrate (Mbps)	1.90	0.25	1.81	1.99	30
Start-up delay (s)	2.88	0.65	2.64	3.13	30

Metric	Mean	Std dev	95% CI low	95% CI high	n
Bitrate switches	4.33	1.83	3.65	5.01	30
Retransmissions	109.7	16.7	103.5	115.9	30
Timeouts	19.4	4.4	17.8	21.0	30
Goodput (Mbps)	3.60	0.28	3.49	3.70	30
Rebuffering time (s)	0.00	0.00	0.00	0.00	30

All 30 runs completed; none was excluded. The QoE interval of ±1.3 points shows that seed-to-seed variation (standard deviation 3.5) is small compared with the 6.0-point gap between UDP + ARQ and TCP-like, so that gap is statistically meaningful, whereas the 1.4-point gap between QUIC-like and TCP-like is not. Rebuffering is zero for every seed, so its interval is degenerate; this is reported rather than hidden.

7.5 Experiment 4: throughput-based vs buffer-based ABR

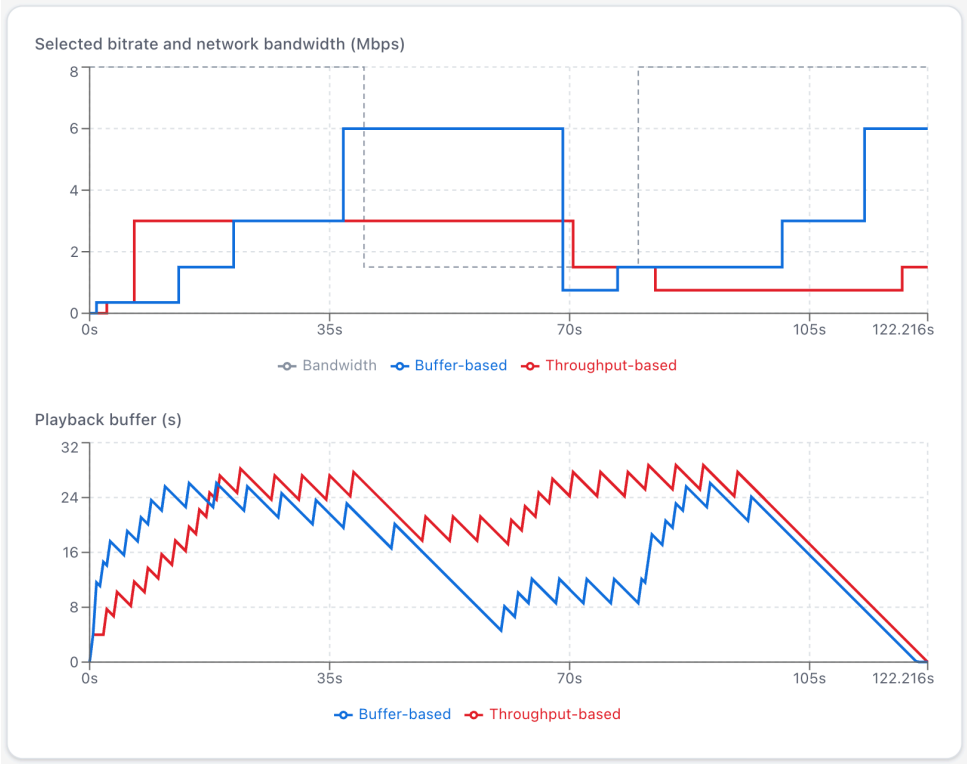


Figure 17. ABR Lab on the "sudden drop" preset – selected bitrate of the two strategies against network bandwidth (seed 1).

Throughput-based — decision log

#	Quality	Rule	Est. Mbps	Buffer s	Size Mb	Download s	Goodput Mbps	Lost	Retx
0	240p (0.35)	startup-initial	—	0.0	1.4	0.30	4.74	0	0
1	720p (3)	throughput-fit	4.74	4.0	12.0	1.82	6.59	34	34
2	720p (3)	throughput-fit	5.51	8.0	12.0	1.55	7.73	1	1
3	720p (3)	throughput-fit	6.10	10.4	12.0	2.31	5.20	4	4
4	720p (3)	throughput-fit	5.84	12.1	12.0	2.18	5.51	2	2
5	720p (3)	throughput-fit	5.77	14.0	12.0	1.86	6.44	2	2
6	720p (3)	throughput-	6.17	16.1	12.0	1.97	6.08	2	2

Figure 18. ABR decision log. Each row shows the rule that fired, the throughput estimate, the buffer level and the reason.

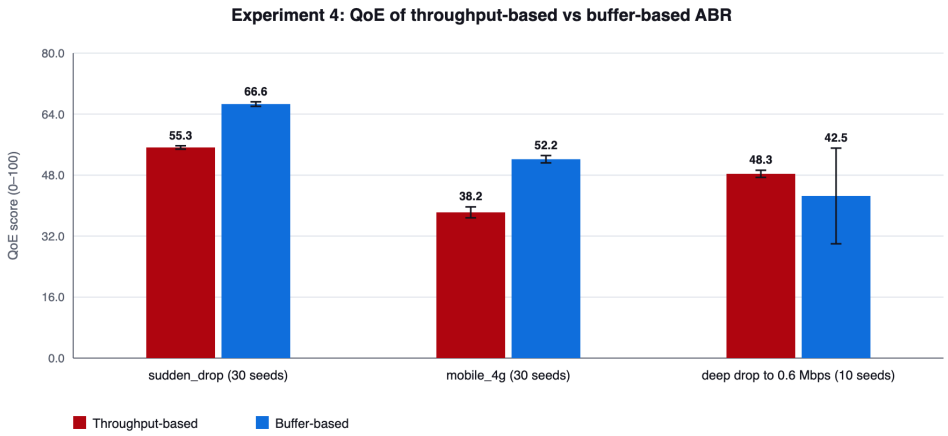


Figure 19. Mean QoE of the two ABR strategies on three traces (95% CI).

Table 12. Experiment 4: ABR comparison (TCP-like transport)

Trace (seeds)	Strategy	QoE	Avg bitrate (Mbps)	Rebuffer (s)	Switches
sudden_drop (30)	Throughput	55.3 ± 0.4	2.03	0.00	4.3
sudden_drop (30)	Buffer	66.6 ± 0.6	3.15	0.00	7.5
mobile_4g (30)	Throughput	38.2 ± 1.4	1.11	0.00	5.4
mobile_4g (30)	Buffer	52.2 ± 1.0	1.72	0.00	7.7
deep drop to 0.6 Mbps (10)	Throughput	48.3 ± 0.9	2.19	1.11	7.1
deep drop to 0.6 Mbps (10)	Buffer	42.5 ± 12.5	3.17	6.79	8.2

On the sudden-drop and mobile-4G presets, buffer-based ABR achieves 11.4 and 13.9 higher QoE respectively. The reason is visible in the logs: under loss, TCP-like goodput is depressed by retransmissions and timeouts, so the harmonic-mean estimate times the 0.8 safety factor selects low rungs even though the link could carry more. Buffer-based ABR is insensitive to this bias because it reacts only to buffer occupancy. The picture reverses on a collapse to 0.6 Mbps: with a full buffer the buffer-based strategy keeps requesting high rungs until the buffer has already drained, stalling for 6.8 s on average against 1.1 s for throughput-based ABR (the wide interval of ±12.5 QoE points on the buffer-based result, with only 10 seeds, means this single comparison is imprecise).

7.6 Experiment 5: sensitivity analysis

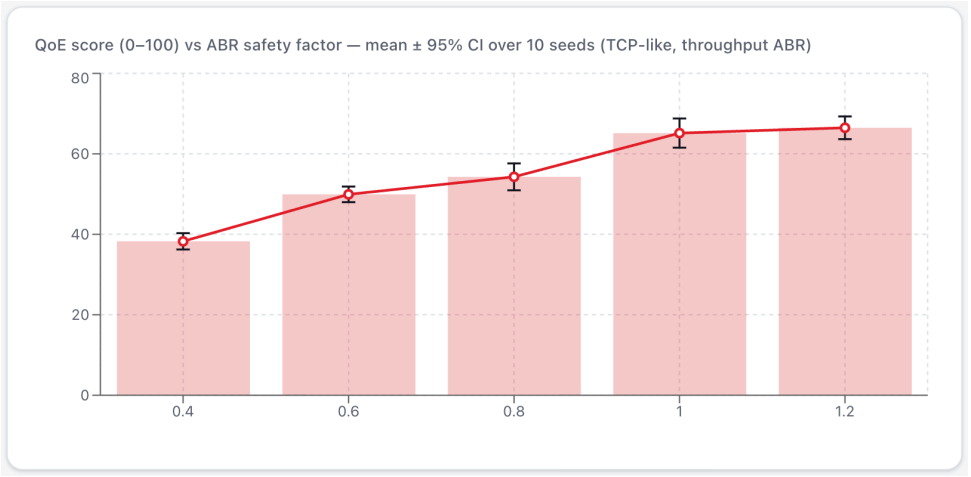


Figure 20. What-If page – QoE against the ABR safety factor (mean ± 95% CI over 10 seeds).

Table 13. Experiment 5: sensitivity of TCP-like / throughput-ABR QoE (20 seeds per value)

Parameter	Value	QoE (mean ± CI)	Avg bitrate (Mbps)	Switches
Safety factor	0.4	37.9 ± 1.4	0.88	2.3
Safety factor	0.6	50.7 ± 1.2	1.46	2.8
Safety factor	0.8	54.4 ± 1.9	1.89	3.9
Safety factor	1	65.4 ± 1.9	2.69	3.0
Safety factor	1.2	67.5 ± 1.5	2.92	2.0
Segment length (s)	2	52.3 ± 1.8	1.79	6.6
Segment length (s)	4	54.4 ± 1.9	1.89	3.9
Segment length (s)	6	51.9 ± 2.2	2.01	3.3
Segment length (s)	8	61.0 ± 2.5	2.19	2.9
Loss probability	0	77.5 ± 2.8	4.06	4.7
Loss probability	0.005	54.4 ± 1.9	1.89	3.9
Loss probability	0.01	46.3 ± 1.3	1.45	3.2
Loss probability	0.03	23.6 ± 2.6	0.69	4.5

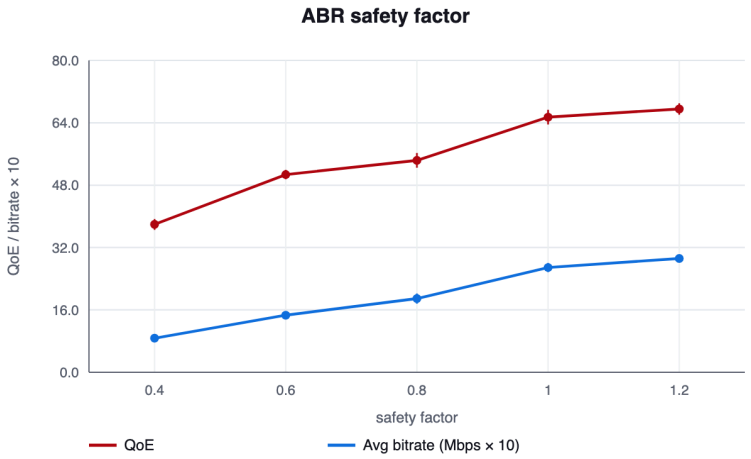


Figure 21. Effect of the ABR safety factor.

- **Safety factor.** QoE rises from 37.9 at 0.4 to 67.5 at 1.2, because a lower factor leaves capacity unused. A more aggressive factor is not punished here since the buffer prevents stalls in this scenario; with a smaller buffer or a harsher trace the penalty would appear, which the What-If page allows the reader to test.

- **Segment duration.** Shorter segments adapt faster but switch more often (6.6 switches at 2 s against 2.9 at 8 s). The QoE differences among 2, 4 and 6 s are within their confidence intervals; 8 s scores highest (61.0); a plausible reason is that fewer, larger downloads pay the per-segment request round trip and timeout risk less often, but the experiment does not isolate that cause.
- **Loss.** QoE falls monotonically from 77.5 at 0 % loss to 23.6 at 3 %, with the average bitrate dropping from 4.06 to 0.69 Mbps: loss reaches the viewer through lower TCP goodput and therefore lower ABR quality, as predicted by the Mathis law.

7.7 Failure injection

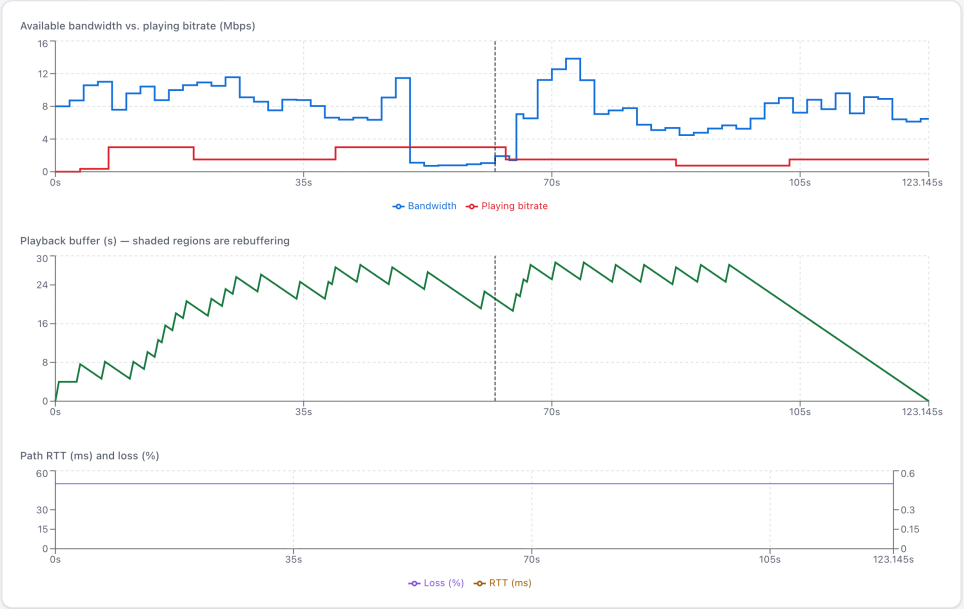


Figure 22. Live Simulation – a 15 s "bandwidth −80 %" fault injected at t = 50 s; the buffer drains while the bandwidth is low and recovers after the fault.

Failure injection re-simulates the session with the same seed and a fault window added to the configuration, so the effect of the fault is isolated from random variation. In this example the throughput-based ABR reacts after the first slow segment, lowers the bitrate and keeps the buffer from emptying; the buffer chart shows the sawtooth shape typical of a client that downloads in bursts and idles when the buffer is full.

7.8 Results page: QoE breakdown and logs



Figure 23. Results page – the QoE score decomposed into quality and penalty components.

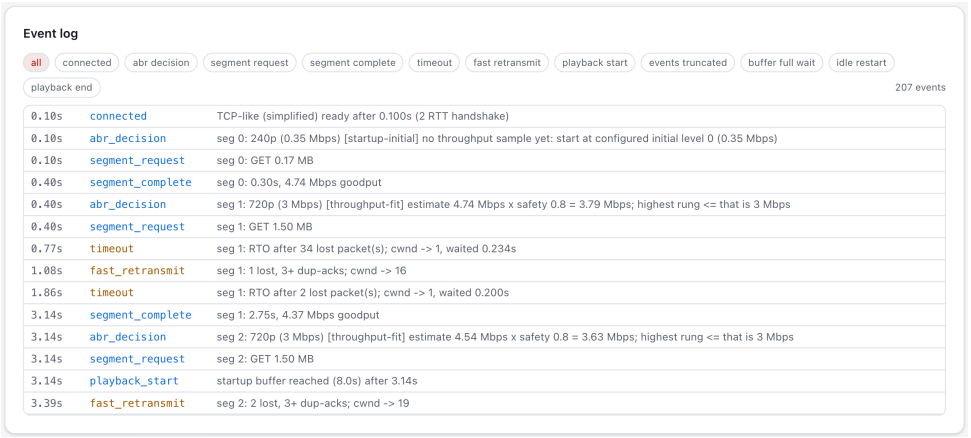


Figure 24. event log – handshake, ABR decisions, requests, timeouts and fast retransmits with timestamps.

7.9 Experiment 6: real socket baseline

A separate script (backend/experiments/socket_experiment.py) was run on the author's machine over the loopback interface and recorded in docs/socket-experiment-results.json. It transferred 50 MB over TCP three times and sent 50000 UDP datagrams three times. TCP connect() completed in 0.07–1.12 ms and delivered every byte; all UDP datagrams arrived (loss 0.0 %, 0 reordered).

These measurements are a baseline only. Loopback has no propagation delay and no random loss, so it shows the absence of the effects the simulator models and cannot validate it. A Wireshark capture with induced loss and delay (macOS dnctl/pfctl or Linux tc netem) was **not** performed; the procedure is documented in docs/socket-experiment.md for future work [25]. No claim of real-network validation is made.

7.10 Discussion

RQ1 – transport comparison. Under identical conditions the transports did not differ in rebuffering (none occurred) but in delivered quality. Reliable delivery (TCP-like, QUIC-like) trades time for completeness; unreliable delivery trades completeness for time; and UDP with ARQ obtains completeness without TCP's ordering constraint and without a congestion window that halves on every loss, which in this scenario produced the best QoE. The conclusion is conditional: this UDP-like sender does not back off, so on a path shared with other flows it would be unfair and its advantage would not be sustainable. HOL blocking is real and measurable (34.0 packet·seconds per session) and is reduced about five-fold by independent streams, but at moderate loss and with a 30 s buffer it is a second-order effect on QoE.

RQ2 – validation. The model follows the $1/(RTT \cdot \sqrt{p})$ law within about 9 % where it should and departs, in the expected direction, where timeouts dominate.

RQ3 – ABR. ABR behaviour depends on what the algorithm observes. Throughput-based ABR couples directly to the transport and inherits its loss-induced bias; buffer-based ABR is robust to that bias but exposed to sudden capacity collapses.

RQ4 – sensitivity. Loss and the ABR safety factor have the largest effect on QoE in this scenario, segment duration mainly trades switching against request overhead.

7.10.1 Key takeaways

1. There is no universal winner: UDP-like delivery scored both worst (no recovery, QoE 38.1) and best (with ARQ, QoE 60.5) depending on application policy.
2. Head-of-line blocking is quantifiable; independent streams cut it by a factor of about five without changing retransmission counts.
3. Loss affects viewers mainly through lower TCP goodput and therefore lower ABR quality, long before it causes stalls.
4. Throughput-based and buffer-based ABR fail in opposite situations: biased estimates under loss versus slow reaction to collapses.

5. Single runs are anecdotal; 30-seed intervals separate real differences from noise.

8 Testing and Verification

The backend has 98 automated tests (pytest); the frontend is type-checked with the TypeScript compiler, built with Vite and linted with oxlint. All backend tests pass and the production build completes without errors. The tests assert relationships rather than the monotonic worsening of every metric, because ABR and transport interactions legitimately produce exceptions.

- **Reproducibility and units:** identical seeds give identical output; 3 Mbps × 4 s = 12 Mbit = 1.5 MB; stepped-trace integration.
- **Buffer and start-up:** growth, depletion, never negative, stall start/end, end of playback is not a stall.
- **ABR and ladder:** rules, safety factor, floor/cap, reservoir/cushion hysteresis; bitrates always on the ladder.
- **Loss and recovery:** retransmissions, HOL blocking (TCP-like > QUIC-like), ARQ and FEC recovery, no UDP retransmission.
- **QoE, statistics, export, API:** formula arithmetic, t-interval by hand, Mathis units, CSV/JSON, HTTP 422 for invalid configuration, edge cases.

Beyond the unit tests, a zero-loss, high-bandwidth scenario was verified to behave sensibly: no retransmissions or stalls, the ladder top reached and QoE above 85.

9 Limitations and Threats to Validity

- **Internal validity.** The TCP-like model is a round-based Reno-like abstraction. Agreement with Mathis within about 10 % in its regime is a sanity check, not proof of fidelity to any real stack. Tail-loss probes, RACK, SACK, CUBIC and BBR are not modelled; this makes tail losses more expensive than in a modern TCP stack.
- **Construct validity.** QoE weights are not calibrated against viewer data, so "better" depends on them. Rebuffering is zero-inflated, so its t-interval is approximate.
- **External validity.** Synthetic traces, independent loss and jitter, one flow, no cross traffic and a single bottleneck. The UDP-like sender policy (3× pacing, no feedback) strongly shapes the UDP results, and per-packet independent jitter overstates reordering.
- **Not measured.** Nothing in the simulator comes from a real network, and no real Netflix traffic was observed. The socket baseline is on loopback and is not a validation.
- **Scale.** Netflix-scale behaviour (millions of concurrent clients, cache hierarchies, server selection) is outside the scope of a single-client model.

10 Conclusion and Future Work

This project built TCPlay, a reproducible simulator and dashboard that connects network conditions to transport behaviour, segmented delivery, playback buffering, ABR decisions and QoE, and used it to compare TCP-like, UDP-like and QUIC-like delivery under identical conditions. Supported by 30-seed confidence intervals and a regime-aware validation against the Mathis model, the results show that transport behaviour matters through quality and start-up more than through stalls in a well-buffered session; that head-of-line blocking is measurable and mostly removable by independent streams; that UDP's outcome is decided by the application's recovery policy; and that ABR strategies fail in complementary ways.

The work also illustrates a methodological point: transport comparisons are only meaningful when the network is held identical, results are repeated over seeds, and the model's assumptions are stated next to its conclusions.

10.0.1 Future work

- Replay public mobile-throughput traces and add bursty (Gilbert-Elliott) loss and correlated jitter.
- Add cross traffic and a shared bottleneck; implement CUBIC/BBR variants and tail-loss probes.
- Add a hybrid (throughput + buffer) ABR and model-predictive control.
- Calibrate QoE weights against a subjective quality dataset and use a perceptual quality metric in place of bitrate.
- Record Wireshark captures with induced loss and delay and compare them quantitatively with the simulator.

References

[1] T. Stockhammer, "Dynamic adaptive streaming over HTTP – Standards and design principles," in Proc. ACM MMSys, San Jose, CA, USA, 2011, pp. 133–144.

[2] M. Seufert, S. Egger, M. Slanina, T. Zinner, T. Hossfeld, and P. Tran-Gia, "A survey on quality of experience of HTTP adaptive streaming," IEEE Commun. Surveys Tuts., vol. 17, no. 1, pp. 469–492, 1st Quart. 2015.

[3] W. Eddy, Ed., "Transmission Control Protocol (TCP)," IETF, RFC 9293, Aug. 2022.

[4] J. Postel, "User Datagram Protocol," IETF, RFC 768, Aug. 1980.

[5] J. Iyengar and M. Thomson, Eds., "QUIC: A UDP-Based Multiplexed and Secure Transport," IETF, RFC 9000, May 2021.

[6] A. Langley et al., "The QUIC transport protocol: Design and Internet-scale deployment," in Proc. ACM SIGCOMM, Los Angeles, CA, USA, 2017, pp. 183–196.

[7] Netflix, Inc., "Open Connect overview," Netflix Open Connect program documentation. [Online]. Available: https://openconnect.netflix.com (accessed Oct. 2026).

[8] V. Jacobson, "Congestion avoidance and control," in Proc. ACM SIGCOMM, Stanford, CA, USA, 1988, pp. 314–329.

[9] M. Allman, V. Paxson, and E. Blanton, "TCP Congestion Control," IETF, RFC 5681, Sep. 2009.

[10] V. Paxson, M. Allman, J. Chu, and M. Sargent, "Computing TCP's Retransmission Timer," IETF, RFC 6298, Jun. 2011.

[11] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, "BBR: Congestion-based congestion control," ACM Queue, vol. 14, no. 5, pp. 20–53, 2016.

[12] J. Iyengar and I. Swett, Eds., "QUIC Loss Detection and Congestion Control," IETF, RFC 9002, May 2021.

[13] M. Mathis, J. Semke, J. Mahdavi, and T. Ott, "The macroscopic behavior of the TCP congestion avoidance algorithm," ACM SIGCOMM Comput. Commun. Rev., vol. 27, no. 3, pp. 67–82, Jul. 1997.

[14] J. Padhye, V. Firoiu, D. Towsley, and J. Kurose, "Modeling TCP throughput: A simple model and its empirical validation," in Proc. ACM SIGCOMM, Vancouver, Canada, 1998, pp. 303–314.

[15] ISO/IEC 23009-1, "Information technology — Dynamic adaptive streaming over HTTP (DASH) — Part 1: Media presentation description and segment formats," International Organization for Standardization, 2022.

[16] J. Jiang, V. Sekar, and H. Zhang, "Improving fairness, efficiency, and stability in HTTP-based adaptive video streaming with FESTIVE," in Proc. ACM CoNEXT, Nice, France, 2012, pp. 97–108.

[17] T.-Y. Huang, R. Johari, N. McKeown, M. Trunnell, and M. Watson, "A buffer-based approach to rate adaptation: Evidence from a large video streaming service," in Proc. ACM SIGCOMM, Chicago, IL, USA, 2014, pp. 187–198.

[18] X. Yin, A. Jindal, V. Sekar, and B. Sinopoli, "A control-theoretic approach for dynamic adaptive video streaming over HTTP," in Proc. ACM SIGCOMM, London, U.K., 2015, pp. 325–338.

[19] H. Mao, R. Netravali, and M. Alizadeh, "Neural adaptive video streaming with Pensieve," in Proc. ACM SIGCOMM, Los Angeles, CA, USA, 2017, pp. 197–210.

[20] F. Dobrian et al., "Understanding the impact of video quality on user engagement," in Proc. ACM SIGCOMM, Toronto, Canada, 2011, pp. 362–373.

[21] S. S. Krishnan and R. K. Sitaraman, "Video stream quality impacts viewer behavior: Inferring causality using quasi-experimental designs," in Proc. ACM IMC, Boston, MA, USA, 2012, pp. 211–224.

[22] T. R. Henderson, M. Lacage, G. F. Riley, C. Dowell, and J. Kopena, "Network simulations with the ns-3 simulator," in ACM SIGCOMM Demonstration, Seattle, WA, USA, 2008.

[23] R. Netravali et al., "Mahimahi: Accurate record-and-replay for HTTP," in Proc. USENIX ATC, Santa Clara, CA, USA, 2015, pp. 417–429.

[24] Student [W. S. Gosset], "The probable error of a mean," Biometrika, vol. 6, no. 1, pp. 1–25, 1908.

[25] Wireshark Foundation, "Wireshark user's guide." [Online]. Available: https://www.wireshark.org/docs/ (accessed Oct. 2026).

Appendix Running the project

```
backend:  cd backend && python3.12 -m venv venv && source venv/bin/activate && pip
install -r requirements.txt
          uvicorn app.main:app --port 8000
frontend: cd frontend && npm install && npm run dev   (http://localhost:5173)
tests:    cd backend && python -m pytest -q   (98 tests)
cli:      python -m app.cli compare-trials --n 30 --out ../results/transports.json
```

All numbers in Section 7 were produced by the command-line interface and are stored in the results/ folder of the project and in docs/socket-experiment-results.json. Re-running the commands in the README regenerates them.